Fiches de Cours

Master IMA

Axel PIGEON
axel.pigeon@univ-tlse3.fr

Université Toulouse III – Paul Sabatier
Années universitaires 2025–2027

4 novembre 2026

Table des matières

I Planification **3**

1 Formalisation de la Planification Classique **4**
1.1 Le cadre classique 4
1.2 Algorithmes et Langages de représentation 7

2 Le Planificateur GRAPHPLAN **10**
2.1 Définitions et fonctionnement 10
2.2 Améliorations de GRAPHPLAN 17

Planification

Chapitre 1

Formalisation de la Planification Classique

Contents

1.1 Le cadre classique 4
1.1.1 États et Actions 4
1.1.2 Plan d'action et problème de planification 6
1.1.3 Propriétés et hypothèses du cadre classique 6
1.2 Algorithmes et Langages de représentation 7
1.2.1 Typologie des approches algorithmiques 7
1.2.2 Formalisme STRIPS et résolution du problème du décor 8
1.2.3 Extension ADL – *Action Description Language* et PDDL 9

La planification est un outil essentiel dans beaucoup de domaines aujourd'hui et est utilisée par tout le monde : que ce soit pour prévoir des tâches à effectuer, suivre une recette de cuisine ou anticiper et prévoir l'évolution d'un programme industriel.
Toutes ces applications peuvent être formalisées et généralisées au sein de la théorie de la planification. Celle-ci cherche à synthétiser et résoudre de manière automatique un *problème de planification* par la production d'un *plan*. Ce dernier peut se voir comme une collection d'actions à effectuer pour modifier l'état courant d'une tâche dans le but d'atteindre un *état cible* appelé *but*. L'exécution de ce plan est donc la réalisation de ces différentes étapes dans le monde pour lequel a été défini le problème.
Nous allons ici formaliser mathématiquement ces concepts pour l'instant abstrait et brièvement étudier la notion de complexité de résolution de tels problèmes.

1.1 Le cadre classique

Commençons donc par définir formellement ce qu'est un problème de planification et les différents objets utilisés pour le résoudre.

1.1.1 États et Actions

Définition (**État – Fluent**) . On appelle *état* un ensemble fini de formules atomiques sans symbole de variable pour un monde défini. Cet ensemble représente un état possible

du problème considéré.
Une formule atomique (suite finie d'éléments d'un alphabet ne contenant aucun symbole de variable) est appelée un *fluent*.

Pour passer d'un état à un autre, on définira des *actions*.

Définition (**Opérateur – Action**) . On appellera *opérateur* o un modèle d'action. Il est défini par deux éléments :

1. son nom,
2. un triplet d'ensembles $\langle \text{pr}, \text{add}, \text{de} \rangle$ finis de formules atomiques représentant respectivement les *préconditions*, les *ajouts* et les *suppression* de fluents opérés par l'opérateur sur l'état courant.

On appellera *action*, notée a, une instance d'un opérateur o (i.e toutes les variables de o sont instanciées à partir des variables/constantes du problème). On notera $\text{Pre}(o), \text{Add}(o)$ et $\text{Del}(o)$ les ensembles pr, add et de de l'opérateur o (idem pour a).

Définition (**Action Applicable**) . Soit E un état et a une action d'un problème de planification. On dira que a est applicable à E si les préconditions de a son vérifiées dans E, formellement, cela revient à dire que :

$$\text{Pre}(a) \subseteq E. \tag{1.1.1}$$

On définira alors l'état résultant E' de l'action de a sur E par :

$$\boxed{E' := E \uparrow a = (E \setminus \text{Del}(a)) \cup \text{Add}(a).} \tag{1.1.2}$$

Exemple Considérons la préparation d'un gâteau nécessitant de mélanger des ingrédients puis de les cuire. Définissons un opérateur permettant de cuire un ingrédient ou une préparation désignée par la variable $?x$:

$$\begin{aligned} \text{Cuire}(?x) = \big\langle\, & \text{Pre} = \{\text{melange}(?x), \text{cru}(?x), \text{four_chaud}\}, \\ & \text{Add} = \{\text{cuit}(?x)\}, \\ & \text{Del} = \{\text{cru}(?x)\} \big\rangle \end{aligned}$$

En instanciant la variable $?x$ avec la constante pate, on obtient l'action close :

$$\begin{aligned} a = \text{Cuire}(\text{pate}) = \big\langle\, & \text{Pre}(a) = \{\text{melange}(\text{pate}), \text{cru}(\text{pate}), \text{four_chaud}\}, \\ & \text{Add}(a) = \{\text{cuit}(\text{pate})\}, \\ & \text{Del}(a) = \{\text{cru}(\text{pate})\} \big\rangle \end{aligned}$$

Soit l'état courant :

$$E = \{\text{melange}(\text{pate}), \text{cru}(\text{pate}), \text{four_chaud}, \text{propre}(\text{moule})\}$$

Puisque $\text{Pre}(a) \subseteq E$, l'action a est applicable. L'état résultant $E' = E \uparrow a$ est :

$$\begin{aligned} E' &= \big(E \setminus \text{Del}(a)\big) \cup \text{Add}(a) \\ &= \{\text{melange}(\text{pate}), \text{four_chaud}, \text{propre}(\text{moule}), \text{cuit}(\text{pate})\} \end{aligned}$$

On note que le fait propre(moule) reste vrai dans E' sans qu'il ait été nécessaire de mentionner son invariance (résolution du problème du décor).

1.1.2 Plan d'action et problème de planification

La résolution d'un problème se fait par l'application séquentielle d'actions applicables sur un état initial E : on appelera cela un *plan séquentiel*.

Définition (**Plan Séquentiel & Application**) . On définit un *plan séquentiel*, P, comme une séquence finie d'actions $\langle a_1, a_2, \ldots, a_n \rangle$. L'application d'un plan P à un état E, notée $E\mathcal{A}P$, est définie par récurrence sur la longueur de P :

- Si $E = \bot$, alors $\bot\mathcal{A}P := \bot$.
- Pour le plan vide : $E\mathcal{A}\langle\rangle := E$.
- Pour $P = \langle a \rangle \cdot P'$ (où a est une action et P' une séquence d'actions) :

$$E\mathcal{A}(\langle a \rangle \cdot P') := \begin{cases} (E \uparrow a)\mathcal{A}P' & \text{si } \mathrm{Prec}(a) \subseteq E, \\ \bot & \text{sinon.} \end{cases} \tag{1.1.3}$$

On peut alors définir le concept de *problème de planification*, noté $\langle \mathcal{O}, I, B \rangle$ tel que :

- $\mathcal{O}$ désigne l'ensemble des opérateurs utilisables dans le domaine considéré. A désigne l'ensemble des actions que l'on peut instancier à partir des opérateurs de $\mathcal{O}$.
- I est l'état initial du problème composé d'un ensemble de fluents.
- B est le but du problème (un état), aussi composé de fluents.

L'objectif de la planification étant de développer un plan séquentiel permettant de résoudre le problème $\langle \mathcal{O}, I, B \rangle$, on appellera un tel plan un *plan solution*.

Définition (**Plan-solution**) . Soit $\mathcal{P} = \langle \mathcal{O}, I, B \rangle$ un problème de planification. Un plan séquentiel P est un *plan-solution* (ou *plan correct*) si et seulement si :

$$I\mathcal{A}P \neq \bot \quad \text{et} \quad B \subseteq (I\mathcal{A}P) \tag{1.1.4}$$

Remarque (Flexibilité) La *flexibilité* d'un plan-solution désigne sa capacité à subir des modifications (modification de l'ordonnancement d'actions non contraintes, instanciation retardée de variables libres) sans perdre sa correction.

1.1.3 Propriétés et hypothèses du cadre classique

La production d'un *plan-solution* sera réalisée par un algorithme de planification, noté Planif. On pourra définir plusieurs propriétés recherchées pour cet algorithme.

Propriété (**Planificateur**) . Soit un algorithme de planification Planif opérant sur un formalisme donné.

- **Sain (ou correct) :** Planif est sain si tout plan qu'il produit pour une instance $\mathcal{P}$ est un plan-solution.
- **Complet :** Planif est complet si, dès lors qu'il existe au moins un plan-solution pour $\mathcal{P}$, il en produit un en temps fini.
- **Systématique :** Planif est systématique si deux feuilles distinctes quelconques de son arbre de recherche correspondent à des plans-solutions distincts (absence de redondance dans l'exploration).
- **Optimal :** Planif est optimal relativement à un critère de coût donné (nombre d'actions, temps d'exécution, parallélisme) si tout plan-solution produit minimise ce critère.

Toute la complexité de la planification réside donc dans la recherche d'un tel algorithme.

Proposition (Hypothèses du cadre classique) Le cadre classique de la planification restreint l'environnement et l'espace d'actions selon six hypothèses :

1. **Fini :** L'ensemble des objets et des fluents est fini.
2. **Statique :** Le monde n'évolue que sous l'effet des actions de l'agent (pas d'événements exogènes).
3. **Observabilité totale :** L'état courant est parfaitement et entièrement connu à chaque étape.
4. **Déterministe :** L'application d'une action a sur un état E produit un unique état résultant prévisible.
5. **Discret :** Les transitions d'états s'effectuent par sauts discrets.
6. **Instantané :** Les actions n'ont pas de durée propre (durée nulle) et ne consomment pas de ressources continues.

La résolution d'un problème de planification par la recherche d'un plan-solution est assez difficile car elle nécessite de définir l'ensemble des objets pour modéliser le problème (états, fluents, actions, etc...). Il faut définir des actions pertinentes en fonction des états choisis et être capable de les décrire au mieux pour permettre un ordonnancement exécutable des plans d'actions générés. Le théorème ci-dessous donne un ordre d'idée de la complexité de cette recherche.

Théorème (**Complexité en cadre classique**) . Pour un problème de planification classique $\mathcal{P} = \langle \mathcal{O}, I, B \rangle$ représenté en logique propositionnelle / STRIPS :

- Le problème de décision de l'*existence d'un plan-solution* (PLANEXIST) est **PSPACE-complet**.
- Le problème de décision de l'*existence d'un plan-solution de longueur inférieure ou égale à* $k \in \mathbb{N}$ (avec k encodé en unaire) est **NP-complet**.

1.2 Algorithmes et Langages de représentation

Cette section introduit les principaux paradygmes de résolution d'un problème de planification. On présentera les différentes méthodes canoniques avant d'introduire les langages de représentation (PDDL, STRIPS) et nous finirons par une introduction à GRAPHLAN avant de le traiter plus en détail dans le chapitre suivant.

1.2.1 Typologie des approches algorithmiques

La résolution effective d'un problème de planification $\mathcal{P} = \langle \mathcal{O}, I, B \rangle$ se heurte à l'explosion combinatoire de l'espace de recherche. Les algorithmes du cadre classique se divisent en deux grandes familles fondamentales.
La première se concentre sur une recherche directe dans les espaces de recherche. Cette approche explore explicitement un graphe orienté où chaque transition correspond à une décision de planification. On distingue deux espaces fondamentaux :

- **Espace des états :** Les nœuds du graphe sont des états complets du monde (voir Figure 1.1). On peut alors trouver un plan-solution par *progression (chaînage avant)* en explorant l'espace depuis l'état initial I et en appliquant des actions applicables jusqu'à atteindre un état contenant B. Ou alors, par *régression (chaînage arrière)* : on part de la spécification du but B et on remonte vers l'état initial I par calcul de préconditions inverses.

- **Espace des plans partiels (POCL – *Partial Order Causal Link*) :** Les nœuds sont des plans incomplets (actions partiellement ordonnées, variables non totalement instanciées et liens causaux). La recherche consiste à éliminer les défauts du plan (préconditions non satisfaites ou conflits causaux) jusqu'à obtenir un plan valide.

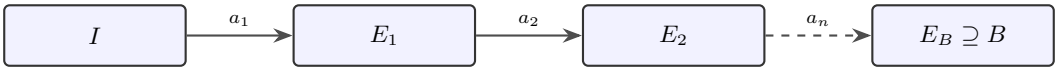


FIGURE 1.1 – Schéma d'un espace d'états d'un problème de planification. On représente les états dans un graphe orienté où chaque arrête traduit l'effet d'une action applicable entre deux états.

Plutôt que d'explorer l'espace d'états de manière séquentielle, les approches par *compilation de plans* (voir Figure 1.2) traduisent le problème de planification de taille ou d'horizon fixé k en une instance d'un problème cible bien caractérisé, bénéficiant ainsi de l'efficacité des solveurs génériques modernes.

- **SATPLAN / BLACKBOX (Compilation propositionnelle) :** Le problème de planification borné à k pas de temps est codé sous la forme d'une formule propositionnelle en forme normale conjonctive (CNF). Un solveur SAT (DPLL, CDCL) recherche une affectation satisfaisable dont le modèle est ensuite décodé en un plan-solution.
- **GP-CSP (Compilation en CSP) :** Les interactions entre fluents et actions sont modélisées sous la forme d'un problème de satisfaction de contraintes (CSP) où les variables représentent les supports d'actions et les contraintes modélisent les exclusions et préconditions.
- **GRAPHPLAN :** Approche intermédiaire qui commence par construire en temps polynomial un graphe compact alternant couches de fluents et d'actions, annoté d'exclusions mutuelles (*mutex*), avant d'y extraire une solution par retour arrière ou compilation.

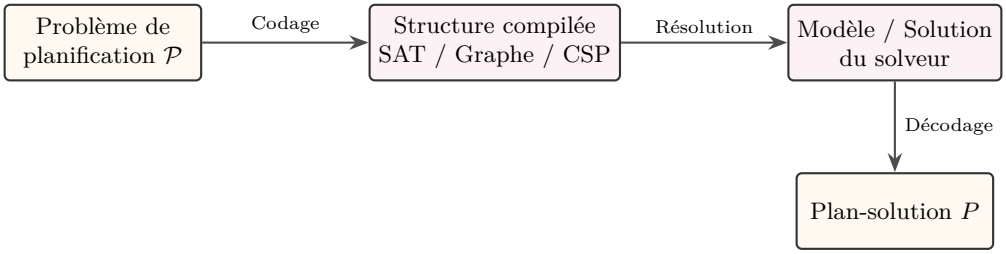


FIGURE 1.2 – Schéma représentant les étapes de construction d'un plan-solution à l'aide d'une méthode par compilation de plans.

La formalisation effective d'un univers de planification impose un compromis fondamental entre le pouvoir expressif du langage de modélisation et la complexité algorithmique du calcul des transitions d'états. L'évolution des formalismes historiques traduit la recherche de cet équilibre.

1.2.2 Formalisme STRIPS et résolution du problème du décor

Le formalisme STRIPS (*Stanford Research Institute Problem Solver*) constitue le noyau minimaliste de la planification classique.

Définition (**Modélisation STRIPS**) . Dans le formalisme STRIPS, le langage est restreint à la logique propositionnelle sans négation explicite :

- Un état $E \subseteq \mathcal{F}$ est une conjonction finie de formules atomiques closes (fluents). Les faits absents de E sont réputés faux en vertu de l'*hypothèse du monde clos*.
- Un opérateur o est défini par le triplet $\langle \text{Prec}(o), \text{Add}(o), \text{Del}(o) \rangle$, où chaque ensemble ne contient que des littéraux atomiques positifs.

Proposition (Résolution du problème du décor – *Frame Problem*) Soit une action a applicable sur un état E (i.e $\text{Prec}(a) \subseteq E$). STRIPS résout le problème du décor par le principe de persistance implicite :

$$\forall f \in E, \quad f \notin \text{Del}(a) \implies f \in (E \uparrow a) \tag{1.2.1}$$

Aucun axiome d'invariance n'est nécessaire : les fluents non mentionnés dans $\text{Del}(a)$ sont conservés par défaut après l'application de a sur l'état E.

Remarque (Verrous expressifs de STRIPS) La simplicité de STRIPS garantit un calcul de transition $E \uparrow a$ en temps linéaire, mais restreint la modélisation. En effet, il devient impossible de formuler des préconditions négatives ou disjonctives. L'impact d'une action sur un état est aussi statique et indépendant de faits périphériques. Enfin, il est impossible de quantifier sur des ensembles : modifier n entités impose de concevoir et planifier n actions unitaires distinctes.

1.2.3 Extension ADL – *Action Description Language* et PDDL

Introduit pour lever la rigidité de STRIPS, ADL enrichit la description des opérateurs en intégrant des fragments de la logique du premier ordre.

Définition (**Caractéristiques du formalisme ADL**) . Dans ADL, un opérateur est caractérisé par un couple $\langle \text{Prec}(o), \text{Eff}(o) \rangle$:

- **Préconditions du premier ordre :** $\text{Prec}(o)$ autorise les connecteurs booléens usuels $(\wedge, \vee, \neg)$, les prédicats d'égalité $(=)$ et les quantificateurs $(\forall, \exists)$.
- **Effets conditionnels :** $\text{Eff}(o)$ accepte des clauses d'implication de la forme :

  $$c \longrightarrow e \tag{1.2.2}$$

  Le littéral e (positif ou négatif) n'est appliqué à l'état que si la condition c était satisfaite avant le déclenchement de l'action.
- **Effets universels :** Des expressions de type $\forall x \, (P(x) \rightarrow e(x))$ permettent de mettre à jour un ensemble arbitraire d'objets en une seule transition.

PDDL - *Planning Domain Description Language* sépare formellement la physique du système de l'instance concrète à résoudre afin de rendre les solveurs indépendants des domaines d'application.

Définition (**Architecture PDDL**) . Une instance PDDL dissocie la modélisation en deux composantes :

1. **Fichier Domaine (`domain.pddl`) :** Spécifie les types d'objets, la liste des prédicats relationnels admis et les schémas d'actions paramétrés.
2. **Fichier Problème (`problem.pddl`) :** Déclare les constantes réelles (objets), l'état initial exhaustif I et la formule logique du but B.

Chapitre 2

Le Planificateur GRAPHPLAN

Contents

2.1 Définitions et fonctionnement 10
2.1.1 Interactions entre actions et Indépendance 10
2.1.2 Exclusions mutuelles (*Mutex*) 12
2.1.3 Algorithme GRAPHPLAN 13
2.2 Améliorations de Graphplan 17
2.2.1 LCGP – Relations d'Autorisation 17
2.2.2 Codages SAT du graphe 18
2.2.3 Codages CSP du graphe 19

2.1 Définitions et fonctionnement

L'algorithme de planification GRAPHPLAN sépare le processus de planification en deux étapes différentes.
La première consiste en la construction d'un graphe de planification représentant les interdépendances entre les états au travers des actions possibles. Cette construction se fait avec une complexité polynômiale en temps et en espace par rapport à la taille du problème posé.
La seconde étape est la recherche d'un solution (s'il en existe) à partir d'un sous-graphe du précédent. Elle produit alors un plan-solution au problème initialement posé.

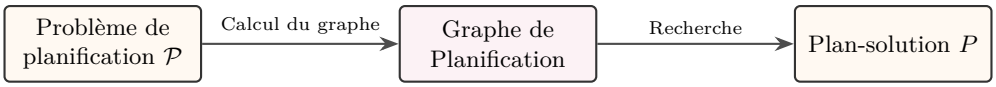


FIGURE 2.1 – Schéma représentant les étapes de l'algorithme GRAPHPLAN pour la recherche d'un plan solution à partir d'un problème de planification énoncé dans le langage STRIPS.

Nous allons ici commencer par poser un formalisme supplémentaire pour traduire les relations d'inter-dépendances dans un tel graphe d'état.

2.1.1 Interactions entre actions et Indépendance

Pour de gros problèmes de planification, tels que des programmes de production industriels ou des projets collaboratifs, il est intéressant de pouvoir exécuter plusieurs actions simultanément dans le graphe de planification. On construit pour cela des ensemble *d'actions indépendantes* Q.

Définition (**Interactions Positives/Négatives**) . Soient deux actions closes a_1 et a_2, on définit deux types d'interactions entre ces deux actions. On dira qu'elles sont en *coopération* ou qu'elles interagissent *positivement* si elles produisent les mêmes effets utiles :

$$\mathrm{Add}(a_1) \cap \mathrm{Add}(a_2) = \emptyset \tag{2.1.1}$$

ou si l'une produit une condition requise par l'autre :

$$\mathrm{Add}(a_1) \cap \mathrm{Prec}(a_2) = \emptyset. \tag{2.1.2}$$

En revanche, on dira qu'elles sont en *conflit* si l'une supprime ce que l'autre ajoute (effet antagoniste) :

$$\mathrm{Add}(a_1) \cap \mathrm{Del}(a_2) \neq \emptyset \quad \text{ou} \quad \mathrm{Add}(a_2) \cap \mathrm{Del}(a_1) \neq \emptyset \tag{2.1.3}$$

ou si l'une supprime une condition nécessaire au déclenchement de l'autre (effet croisé) :

$$\mathrm{Del}(a_1) \cap \mathrm{Prec}(a_2) \neq \emptyset \quad \text{ou} \quad \mathrm{Del}(a_2) \cap \mathrm{Prec}(a_1) \neq \emptyset. \tag{2.1.4}$$

L'objectif sera de limiter au plus les effets conflits entre actions pour favoriser les interactions positives en vue de construire un plan.

Définition (**Indépendance**) . On dira que deux actions a_1 et a_2 sont *indépendantes*, noté $a_1 \# a_2$, si elles n'ont pas d'interactions négatives (conflits) :

$$\mathrm{Del}(a_1) \cap (\mathrm{Prec}(a_2) \cup \mathrm{Add}(a_2)) = \emptyset \quad \text{et} \quad \mathrm{Del}(a_2) \cap (\mathrm{Prec}(a_1) \cup \mathrm{Add}(a_1)) = \emptyset. \tag{2.1.5}$$

On dira que Q est un ensemble d'actions indépendantes si toutes ses actions sont deux à deux indépendantes.

Propriété (**Indépendance**) . Soient deux actions a_1 et a_2 applicables sur un état E. Si a_1 et a_2 sont indépendantes, alors leur ordre d'application sur E n'a pas d'importance :

$$(E \uparrow a_1) \uparrow a_2 = (E \uparrow a_2) \uparrow a_1 = E \uparrow \{a_1, a_2\}. \tag{2.1.6}$$

L'intérêt de travailler avec des ensembles d'opérations indépendantes est de pouvoir toutes les exécuter à la fois dans uen couche du graphe.

Définition (**Action de maintien – *NoOp***) . Soit $f \in \mathcal{F}$ un fluent atomique. On appelle *action de maintien* (ou *NoOp*, pour *No Operation*), notée N_f, l'action triviale définie par :

$$N_f = \big\langle \mathrm{Prec}(N_f) = \{f\}, \quad \mathrm{Add}(N_f) = \{f\}, \quad \mathrm{Del}(N_f) = \emptyset \big\rangle \tag{2.1.7}$$

Dans le graphe de planification, les NoOps assurent deux fonctions fondamentales :

1. *Transport d'invariance :* modéliser explicitement la persistance d'un fait d'une couche F_{i-1} vers la couche suivante F_i sans altérer l'état du monde.
2. *Propagation des conflits :* soumettre la conservation d'un fait aux mêmes règles d'indépendance et d'exclusion mutuelle que les actions standard (notamment avec toute action qui supprime ce fait).

Proposition Soit Q un ensemble d'actions indépendantes et E un état. Alors Q est *applicable* sur E ssi :

$$\bigcup_{a_i \in Q} \mathrm{Prec}(a_i) \subseteq E. \tag{2.1.8}$$

L'état résultant de l'application de tous les $a_i \in Q$ sur E sera caractérisé par :

$$E \uparrow Q = \left(E \setminus \bigcup_{a_i \in Q} \mathrm{Del}(a_i) \right) \cup \left(\bigcup_{a_i \in Q} \mathrm{Add}(a_i) \right). \tag{2.1.9}$$

On peut alors appliquer un ensemble d'actions indépendantes à un état donné en une fois. On définira alors des *plans d'ensembles d'actions indépendantes* $P = \langle Q_1, Q_2, \ldots, Q_n \rangle$ où les Q_i sont des actions indépendantes. Leur application se fera donc successivement sur un état itinial E comme illustré en Figure 2.2.

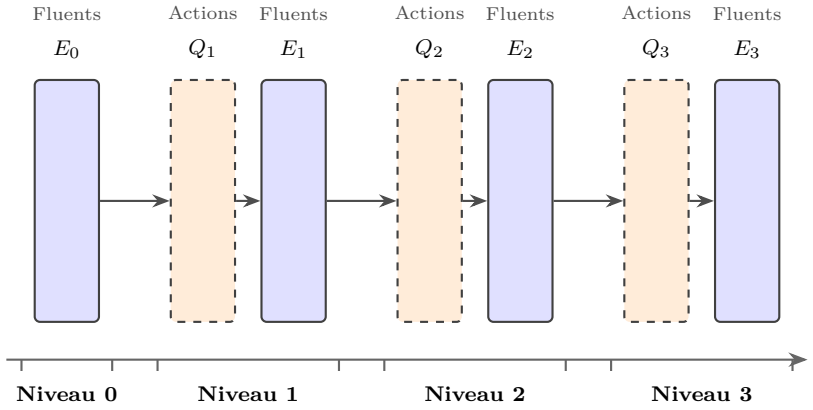


FIGURE 2.2 – Structure alternée par niveaux dans un plan d'ensembles d'actions. Chaque niveau $i \geqslant 1$ applique simultanément un ensemble d'actions indépendantes Q_i sur l'état courant pour produire le nouvel état de fluents E_i.

2.1.2 Exclusions mutuelles (*Mutex*)

Le graphe de planification sur-approximant l'ensemble des états atteignables, des relations d'exclusion mutuelle (*mutex*) sont introduites à chaque niveau pour contraindre la recherche et élaguer les combinaisons physiquement irréalisables. La relation est symétrique et binaire et est définie aussi bien sur les actions que sur les fluents.

Définition (**Exclusion mutuelle entre actions**) . Soient deux actions a_1, a_2 appartenant au même niveau d'actions i. Les actions a_1 et a_2 sont dites *mutuellement exclusives* au niveau i, noté $\mathrm{Mutex}(a_1, a_2)$, si et seulement si l'une des deux conditions suivantes est vérifiée :

1. **Non-indépendance (conflit direct) :** Les actions a_1 et a_2 ne sont pas indépendantes ($a_1 \# a_2$), c'est-à-dire :

$$\mathrm{Del}(a_1) \cap \big(\mathrm{Prec}(a_2) \cup \mathrm{Add}(a_2) \big) \neq \emptyset \quad \text{ou} \quad \mathrm{Del}(a_2) \cap \big(\mathrm{Prec}(a_1) \cup \mathrm{Add}(a_1) \big) \neq \emptyset \tag{2.1.10}$$

2. **Besoins concurrents (conflit hérité) :** Il existe au moins une paire de préconditions mutuellement exclusives au niveau de fluents précédent F_{i-1} :

$$\exists (p, q) \in \mathrm{Prec}(a_1) \times \mathrm{Prec}(a_2) \quad \text{tel que} \quad \mathrm{Mutex}(p, q) \text{ dans } F_{i-1} \tag{2.1.11}$$

Définition (**Exclusion mutuelle entre fluents**) . Soient deux fluents $p, q \in F_i$ au niveau de fluents i. Les fluents p et q sont dits *mutuellement exclusifs* au niveau i, noté $\text{Mutex}(p, q)$, si et seulement si **toutes** les paires d'actions susceptibles de les produire simultanément à ce niveau sont en exclusion mutuelle :

$$\forall (a_1, a_2) \in A_i \times A_i, \quad \big(p \in \text{Add}(a_1) \wedge q \in \text{Add}(a_2)\big) \implies \text{Mutex}(a_1, a_2) \tag{2.1.12}$$

Autrement dit, p et q ne sont pas mutex dans F_i dès lors qu'il existe au moins une paire d'actions (a_1, a_2) non mutex à l'étape i telle que $p \in \text{Add}(a_1)$ et $q \in \text{Add}(a_2)$.

On pourra aussi définir deux types de mutex :

- *Les mutex structurels* : produits lorsqu'une action supprime des préconditions ou des ajouts d'une autre action. Elles seront alors toujours en mutex dans chaque couche.
- *Les mutex contextuels* : lorsque deux actions ont des mutex comme préconditions.

Remarque (Rôle des actions de persistance) Une action de maintien $N_f = \langle \{f\}, \{f\}, \emptyset \rangle$ est traitée comme une action usuelle. Par conséquent, si une action $a \in A_i$ supprime le fluent f ($f \in \text{Del}(a)$), elle entre directement en conflit de non-indépendance avec N_f car $\text{Del}(a) \cap \text{Add}(N_f) = \{f\} \neq \emptyset$. Cela garantit qu'un fluent f et sa négation deviennent automatiquement mutex au niveau suivant.

2.1.3 Algorithme GRAPHPLAN

Proposé par Blum et Furst (1995), GRAPHPLAN résout le problème de planification en découplant la recherche en deux phases complémentaires :

1. **Phase d'extension :** construction alternée de couches de fluents F_i et de couches d'actions A_i annotées de relations d'exclusion mutuelle. Cette phase est polynomiale en temps et en espace.
2. **Phase d'extraction :** recherche arrière par retour-arrière (*backtracking*) dans le sous-graphe pour identifier un plan valide. La complexité de cette phase est NP-Complète.

Le détail de l'algorithme est donné en Algorithme 1. Mais pour son application pratique à la main, la proposition suivante en détaille les étapes.

Proposition (Algorithme GRAPHPLAN) Le tracé manuel d'un graphe de planification s'effectue par colonnes alternées de gauche à droite. On commence par construire le graphe en développant les couches successivement et en calculant les mutex entre chaque couches. Une fois l'état final découvert, on peut tenter l'extraction d'un plan solution.

1. **Construction du graphe** :
   (a) **Initialisation du niveau 0 (F_0)** :
      - Lister verticalement tous les fluents de l'état initial : $F_0 = I$.
      - *Règle d'or :* Il n'y a *jamais* de relation d'exclusion mutuelle dans la couche initiale F_0.

   (b) **Construction de la couche d'actions (A_k depuis F_{k-1}) :**
      - *Actions de maintien (NoOps) :* Dessiner immédiatement une action N_f pour chaque fait $f \in F_{k-1}$.
      - *Actions du domaine :* Instancier une action $a \in \mathcal{A}$ si et seulement si tous ses fluents préconditions sont présents dans F_{k-1} et qu'aucune paire $(p, q) \subseteq \text{Prec}(a)$ n'est marquée par un arc mutex dans F_{k-1}.
      - Tracer les arcs orientés reliant chaque précondition dans F_{k-1} à son action dans A_k.

(c) **Audit des mutex d'actions dans A_k :** Pour chaque paire non ordonnée d'actions distinctes $\{a_1, a_2\} \subseteq A_k$, vérifier les deux conditions suivantes :

- *Conflit direct (non-indépendance) :* Vérifier si $\text{Del}(a_1) \cap (\text{Prec}(a_2) \cup \text{Add}(a_2)) \neq \emptyset$ ou symétriquement pour a_2. (Dès qu'une action a supprime un fluent f ($f \in \text{Del}(a)$), elle est *immédiatement* mutex avec le NoOp N_f car $\text{Del}(a) \cap \text{Add}(N_f) = \{f\} \neq \emptyset$.)
- *Besoins concurrents (conflit hérité) :* Vérifier s'il existe une paire $(p, q) \in \text{Prec}(a_1) \times \text{Prec}(a_2)$ telle que $\text{Mutex}(p, q)$ dans F_{k-1}.
- Si au moins une condition est satisfaite, tracer un lien d'exclusion mutuelle entre a_1 et a_2.

(d) **Construction de la couche de fluents (F_k depuis A_k) :**

- Reporter dans F_k tous les fluents produits par les effets positifs (Add) des actions et des NoOps de A_k.
- Relier chaque action de A_k à ses effets positifs dans F_k (et optionnellement marquer les effets de suppression Del en pointillés).

(e) **Audit systématique des mutex de fluents dans F_k :** Pour chaque paire de fluents distincts $\{p, q\} \subseteq F_k$:

- Identifier l'ensemble de tous les couples d'actions $(a_1, a_2) \in A_k \times A_k$ susceptibles de les générer conjointement ($p \in \text{Add}(a_1)$ et $q \in \text{Add}(a_2)$).
- Si *toutes* les paires de générateurs sont reliées par un mutex dans A_k, alors p et q sont déclarés mutuellement exclusifs dans F_k.
- Dès qu'il existe *au moins un couple* non mutex d'actions productrices, le conflit saute : p et q ne sont pas mutex.

2. **Phase d'extraction pas-à-pas** : Dès que $B \subseteq F_k$ et qu'aucun couple de buts n'est mutex dans F_k, basculer en recherche arrière.

(a) Poser le sous-but initial au niveau final : $G_k = B$.

(b) Pour chaque niveau i allant de k à 1 :

- Pour chaque fluent $g \in G_i$, sélectionner *une* action productrice $a \in A_i$ (en favorisant les NoOps en premier lieu pour limiter les effets de bord).
- Vérifier que l'ensemble d'actions sélectionné Q_i ne contient *aucune paire en exclusion mutuelle*. Si un conflit survient, faire un retour-arrière (*backtrack*) et tester une combinaison alternative d'actions productrices.
- Calculer le nouveau sous-but à satisfaire au niveau précédent :

$$G_{i-1} = \bigcup_{a \in Q_i} \text{Prec}(a) \tag{2.1.13}$$

(c) Si le retour-arrière atteint le niveau 0 avec $G_0 \subseteq I$, la séquence $\langle Q_1, Q_2, \ldots, Q_k \rangle$ constitue le plan-solution. En supprimant les NoOps de chaque ensemble Q_i, on obtient les actions exécutables du plan.

Exemple (Exemple canonique) Considérons le domaine classique composé des trois actions suivantes et des NoOps associés sur $\{a, b, c, d\}$:

$$A : a \longrightarrow +b, \qquad B : a \longrightarrow +c - a, \qquad C : b \wedge c \longrightarrow +d$$

Soit l'état initial $I = \{a\}$ et le but $G = \{d\}$. Le déroulement met en évidence les mécanismes clés :

- Au niveau A_1, l'action B supprime le fluent a ($\text{Del}(B) = \{a\}$) tandis que N_a l'ajoute ($\text{Add}(N_a) = \{a\}$). Ces actions sont donc en conflit direct : $\text{Mutex}(B, N_a)$.

- Dans F_1, les fluents a et c ne peuvent être produits conjointement que par le couple (N_a, B). Ce couple étant mutex, les fluents héritent de l'exclusion : Mutex(a, c) dans F_1.
- L'action C requiert b et c. Elle ne peut pas être déclenchée valablement au niveau 2 si b et c proviennent d'actions incompatibles, mais peut être satisfaite dès que les paires de supports deviennent mutuellement indépendantes.
- *Plan extrait :* l'extraction arrière produit le plan d'ensembles d'actions :

$$P = \langle Q_1 = \{A, N_a\}, \quad Q_2 = \{N_b, B\}, \quad Q_3 = \{C\} \rangle \tag{2.1.14}$$

Ce plan, une fois post-traité et débarrassé des actions de maintien triviales, fournit la séquence exécutable minimale :

$$\langle A, B, C \rangle \tag{2.1.15}$$

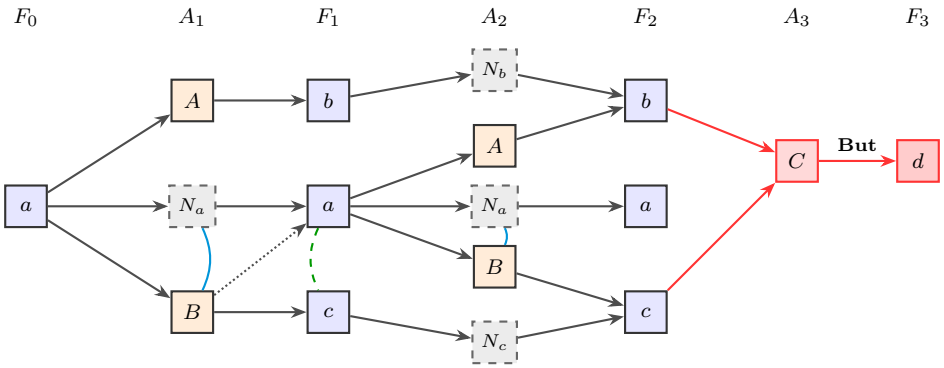


FIGURE 2.3 – Représentation du graphe de planification pour le domaine canonique. Les traits pleins bleus représentent les mutex d'actions, les pointillés verts les mutex de fluents. Le but d est atteint et extrait au niveau 3.

Une caractéristique fondamentale de GRAPHPLAN est que le calcul des exclusions mutuelles est strictement restreint à une *arité 2* (incompatibilités par paires) afin de préserver la complexité polynomiale de la phase d'extension.

Remarque (Sur-approximation et pannes de consistance globale) L'absence de mutex binaire entre tous les éléments d'un sous-ensemble n'implique pas la réalisabilité physique conjointe de ce sous-ensemble. Soient trois actions ou trois fluents mutuellement compatibles par paires :

$$\neg\,\text{Mutex}(x_1, x_2), \quad \neg\,\text{Mutex}(x_2, x_3), \quad \neg\,\text{Mutex}(x_1, x_3) \tag{2.1.16}$$

Bien qu'aucun mutex binaire n'interdise leur sélection conjointe, il peut être structurellement impossible de satisfaire simultanément la conjonction $x_1 \wedge x_2 \wedge x_3$ au sein d'un même état valide. Le graphe de planification constitue ainsi une *sur-approximation optimiste* de l'espace d'atteignabilité. C'est cette insuffisance intrinsèque des contraintes binaires qui justifie le caractère NP-complet de la phase d'extraction arrière, celle-ci devant effectuer un retour-arrière complet pour vérifier la consistance globale du plan.

Algorithme 1 : Algorithme GRAPHPLAN

Entrées : Problème de planification $\mathcal{P} = \langle \mathcal{F}, \mathcal{A}, I, B \rangle$
Sortie : Un plan d'ensembles d'actions $P = \langle Q_1, \ldots, Q_k \rangle$, ou **échec**

```
// Initialisation du niveau 0
```
$k \leftarrow 0$
$F_0 \leftarrow I$
$\text{Mutex}(F_0) \leftarrow \emptyset$
$\mathcal{M} \leftarrow \emptyset$ // Table de mémorisation (nogoods)
while ***vrai*** **do**
  // Condition de déclenchement de la recherche arrière
  if $B \subseteq F_k$ ***et*** $\forall (g_1, g_2) \in B \times B,\ \neg \text{Mutex}(g_1, g_2 \mid F_k)$ **then**
    $P \leftarrow \texttt{ExtrairePlan}(B, k, \mathcal{M})$
    if $P \neq$ ***échec*** **then**
      return P
  // Test de stabilisation (incomplétude prouvée)
  if $k > 0$ ***et*** $F_k = F_{k-1}$ ***et*** $\text{Mutex}(F_k) = \text{Mutex}(F_{k-1})$ **then**
    return échec // Le graphe a stationné sans solution
  // Phase d'extension vers le niveau $k+1$
  $k \leftarrow k + 1$
  $A_k \leftarrow \{N_f \mid f \in F_{k-1}\} \cup \{a \in \mathcal{A} \mid \text{Prec}(a) \subseteq F_{k-1} \wedge \forall p, q \in \text{Prec}(a), \neg \text{Mutex}(p, q \mid F_{k-1})\}$
  Calculer $\text{Mutex}(A_k)$ via conflits directs et préconditions concurrentes
  $F_k \leftarrow \bigcup_{a \in A_k} \text{Add}(a)$
  Calculer $\text{Mutex}(F_k)$ via paires d'actions productrices exclusives
Procédure $\texttt{ExtrairePlan}(G, i, \mathcal{M})$ **:**
  if $i = 0$ **then**
    return $\langle\rangle$ // Succès : le niveau initial est atteint
  if $(i, G) \in \mathcal{M}$ **then**
    return échec // Sous-but déjà exploré et prouvé stérile
  foreach *ensemble non mutex* $Q \subseteq A_i$ *tel que* $G \subseteq \bigcup_{a \in Q} \text{Add}(a)$ **do**
    $G' \leftarrow \bigcup_{a \in Q} \text{Prec}(a)$
    $P' \leftarrow \texttt{ExtrairePlan}(G', i-1, \mathcal{M})$
    if $P' \neq$ ***échec*** **then**
      return $P' \oplus \langle Q \rangle$
  $\mathcal{M} \leftarrow \mathcal{M} \cup \{(i, G)\}$ // Mémorisation du conflit (nogood)
  return échec

2.2 Améliorations de Graphplan

Bien que la phase d'extension du graphe soit de complexité polynomiale, l'extraction de solution reste un problème NP-complet. Deux axes majeurs d'optimisation ont été développés dans la littérature. La première consiste en une optimisation structurelle du graphe et des exclusions mutuelles via une relaxation de la condition d'indépendance grâce à la *relation d'autorisation* (LCGP). La seconde approche utilise plutôt des traductions du problème de planification en problème de satisfiabilité booléenne (SAT) ou de réseau de contraintes (CSP) afin de tirer parti des algorithmes de résolution modernes.

2.2.1 Lcgp – Relations d'Autorisation

Développé par Cayrol, Régnier et Vidal (2000), le planificateur LCGP (*Linearization of Chains in Graphplan*) repose sur un constat : l'indépendance stricte $a_1 \# a_2$ exigée par GRAPHPLAN pour exécuter deux actions dans une même couche est trop conservative. Deux actions n'ont pas besoin d'être symétriquement indépendantes dès lors qu'il existe un ordre d'exécution séquentiel valide au sein de l'étape sans détruire leurs effets conjoints.

Définition (**Relation d'autorisation**) . Soient deux actions a_1 et a_2. On dit que a_1 *autorise* a_2, noté $a_1 \angle a_2$, si et seulement si l'exécution préalable de a_1 ne détruit ni les préconditions de a_2, ni ses effets d'ajout :

$$a_1 \angle a_2 \iff \begin{cases} \mathrm{Del}(a_1) \cap \mathrm{Prec}(a_2) = \emptyset \\ \mathrm{Del}(a_2) \cap \mathrm{Add}(a_1) = \emptyset \end{cases} \tag{2.2.1}$$

Deux actions sont dites *autorisées* si $a_1 \angle a_2$ ou $a_2 \angle a_1$.

Remarque (Comparaison formelle avec l'indépendance) La relation d'indépendance classique correspond à la conjonction symétrique des deux autorisations :

$$a_1 \# a_2 \iff (a_1 \angle a_2) \wedge (a_2 \angle a_1) \tag{2.2.2}$$

La relation d'autorisation est donc une relaxation stricte de l'indépendance.

Définition (**Ensemble d'actions autorisées**) . Un ensemble d'actions Q est un *ensemble autorisé* s'il existe une permutation (linéarisation) $(a_{\sigma(1)}, a_{\sigma(2)}, \ldots, a_{\sigma(m)})$ de ses éléments telle que chaque action autorise les suivantes :

$$\forall j < k, \quad a_{\sigma(j)} \angle a_{\sigma(k)} \tag{2.2.3}$$

Un tel ensemble Q est applicable sur un état E dès lors que $\bigcup_{a \in Q} \mathrm{Prec}(a) \subseteq E$, produisant l'état cumulé identique $E \uparrow Q$.

La relaxation induite par LCGP modifie directement la détection des mutex au niveau des actions :

- Dans GRAPHPLAN : a_1 et a_2 sont mutex si et seulement si elles ne sont pas indépendantes, c'est-à-dire : $\neg(a_1 \angle a_2) \vee \neg(a_2 \angle a_1)$.
- Dans LCGP : a_1 et a_2 sont mutex si et seulement si *aucune n'autorise l'autre* :

$$\mathrm{Mutex}_{\mathrm{LCGP}}(a_1, a_2) \iff \neg(a_1 \angle a_2) \wedge \neg(a_2 \angle a_1) \tag{2.2.4}$$

Différentes conséquences pratiques peuvent alors être observées :

1. *Moins de contraintes d'exclusion :* Le nombre d'arcs mutex diminue drastiquement dans chaque couche d'actions, ce qui permet de compresser les étapes et d'atteindre le but en moins de niveaux.
2. *Extraction de la solution :* LCGP extrait une séquence d'ensembles autorisés, puis applique un algorithme polynomial (notamment l'algorithme PRF) pour réordonner les actions et retourner un plan séquentiel valide.
3. *Gains empiriques :* Sur les domaines peu contraints (tels que *Logistics*), LCGP est en moyenne jusqu'à 1800 fois plus rapide que GRAPHPLAN.

2.2.2 Codages SAT du graphe

Plutôt que d'extraire la solution par retour-arrière classique dans le graphe, l'approche initiée par BLACKBOX (Kautz et Selman, 1999) compile le graphe de planification jusqu'au niveau k en une formule propositionnelle en forme normale conjonctive (CNF). Un solveur SAT complet tente alors de trouver une affectation satisfaisable :

- Si un modèle propositionnel est identifié, il est décodé directement en un plan-solution valide.
- Si la formule est insatisfiable (UNSAT), aucun plan de longueur $\leqslant k$ n'existe ; le graphe est alors étendu au niveau $k+1$ et le processus réitéré.

Définition (Règles de codage SAT – Kautz & Selman, 1999) . Soit GP_k le graphe de planification développé jusqu'au niveau k. Le problème de planification est traduit en la conjonction des clauses propositionnelles suivantes :

1. *Assertions d'état initial et de but :*

$$\left(\bigwedge_{n_f \in \text{NoeudsInit}(GP_k)} n_f\right) \wedge \left(\bigwedge_{n_f \in \text{NoeudsBut}(GP_k)} n_f\right) \tag{2.2.5}$$

2. *Implication d'action vers préconditions :* Si une action est vraie à l'étape i, chacune de ses préconditions doit être vraie à l'étape $i-1$:

$$\bigwedge_{(n_f, n_a) \in \text{ArcsPrec}(GP_k)} (n_a \Longrightarrow n_f) \quad \equiv \quad \bigwedge_{(n_f, n_a) \in \text{ArcsPrec}(GP_k)} (\neg n_a \vee n_f) \tag{2.2.6}$$

3. *Axiomes d'explication des fluents (Frame-axiomes causaux) :* Un fluent (hors état initial) ne peut être vrai à l'étape i que si au moins une action (normale ou NoOp) l'a produit à cette étape :

$$\bigwedge_{n_f \in \text{NoeudsF}(GP_k) \setminus \text{NoeudsInit}(GP_k)} \left(n_f \Longrightarrow \bigvee_{(n_a, n_f) \in \text{ArcsAdd}(GP_k)} n_a\right) \tag{2.2.7}$$

4. *Exclusions mutuelles entre actions :* Deux actions en conflit direct ou hérité ne peuvent pas être sélectionnées simultanément au même niveau :

$$\bigwedge_{\{n_a, n_b\} \in \text{MutexA}(GP_k)} (\neg n_a \vee \neg n_b) \tag{2.2.8}$$

Remarque (Rôle des NoOps dans le codage SAT) Les actions de maintien N_f font partie intégrante de ArcsAdd(GP_k). L'implication $n_f \Longrightarrow \bigvee n_a$ encode donc élégamment à la fois la causalité d'une action réelle et la persistance par défaut, évitant l'explosion combinatoire des frame-axiomes classiques.

2.2.3 Codages CSP du graphe

Développée dans des systèmes comme GPCSP (Kambhampati et Nigenda, 2000), la formulation CSP transforme l'extraction arrière en un problème de satisfaction de contraintes à variables discrètes.

Définition (**Variables & Domaines**) . Pour un horizon k donné, le réseau de contraintes est structuré de façon inverse à la logique des fluents :

- Variables du CSP : chaque fluent présent dans le graphe constitue une variable $f \in \{f_1, f_2, \ldots, f_m\}$.
- Domaine des variables : pour chaque fluent f, la valeur qui lui est affectée représente *l'action productrice* choisie pour l'établir :

$$D(f) = \{a \in \mathcal{A} \cup \text{NoOps} \mid f \in \text{Add}(a)\} \cup \{\bot\} \tag{2.2.9}$$

  où la valeur spéciale $\bot$ indique que le fluent f est inactif (non requis) dans le plan extrait.
- Cas particulier du but : Pour tout fluent $g \in B$ au niveau terminal, le domaine est restreint à ses seules actions productrices : $\bot \notin D(g)$.

On définit alors un *réseau de contraintes* entre ces variables.

Proposition (Contraintes) Le CSP impose trois familles de contraintes sur ces variables :

1. **Contraintes d'activité (flux causal) :** Si un fluent f est soutenu par une action a ($f = a$), alors toutes les préconditions $p \in \text{Prec}(a)$ doivent nécessairement être activées au niveau précédent :
$$(f = a) \Longrightarrow \bigwedge_{p \in \text{Prec}(a)} (p \neq \bot) \tag{2.2.10}$$
2. **Contraintes d'actions mutex :** Si deux actions a_1 et a_2 sont en exclusion mutuelle au même niveau, elles ne peuvent pas être sélectionnées simultanément pour soutenir deux fluents distincts f_1 et f_2 :
$$\text{Mutex}(a_1, a_2) \Longrightarrow \neg\big((f_1 = a_1) \wedge (f_2 = a_2)\big) \tag{2.2.11}$$
   ce qui s'écrit sous forme binaire : $(f_1 = a_1) \Longrightarrow (f_2 \neq a_2)$.
3. **Contraintes de fluents mutex :** Deux fluents mutuellement exclusifs ne peuvent pas être simultanément actifs au même pas temporel :
$$\text{Mutex}(f_1, f_2) \Longrightarrow \neg\big((f_1 \neq \bot) \wedge (f_2 \neq \bot)\big) \tag{2.2.12}$$

L'avantage déterminant du codage CSP est qu'il permet d'exploiter les heuristiques génériques d'ordonnancement de variables et de valeurs guidées par la topologie du graphe :

- Choix des variables (ordre d'instanciation des fluents) :
  - *Level-based :* Sélectionner en priorité les fluents apparaissant aux plus hauts niveaux du graphe (les plus proches du but).
  - *Most constrained (Number of mutex) :* En cas d'égalité de niveau, choisir le fluent possédant le plus grand nombre d'exclusions mutuelles.
- Choix des valeurs (ordre de sélection des actions productrices) :
  - *NoOps-first :* Tenter d'abord les actions de persistance afin de minimiser le nombre d'actions effectives du plan.

- *Lowest appearance level & Least mutex :* Privilégier les actions apparues le plus tôt dans le graphe et ayant le moins d'arcs mutex, maximisant ainsi les chances d'extension sans conflit.

L'utilisation conjointe de ces heuristiques permet à GPCSP d'être compétitif avec les meilleurs solveurs SAT (comme BLACKBOX) tout en consommant une empreinte mémoire nettement inférieure.